Composition recipes — use AIDA *with* these, not instead of

AIDA positioning · best-effort comparison — live source: github.com/joemooney/aida

rendered 2026-07-10

Last updated: 2026-07-09

Most of the "should I use AIDA or X?" questions in this directory have the same real answer: **both.** AIDA is the layer *above* — the durable, vendor-neutral, cross-session graph of intent — and almost every neighbor tool is something you run *inside* that layer, not as a replacement for it. Spec Kit scaffolds a feature; AIDA keeps the feature in a cross-cutting graph. Agent Teams coordinates a burst of agents in one session; AIDA remembers what they did after the session ends. An MCP-speaking editor queries specs; AIDA is the store it queries.

This page is the concrete recipe book: what to layer, in what role, and exactly where the seam is. Where a bridge is still manual today, this page says so — no recipe here depends on tooling that doesn't exist yet.

> **The mental model:** AIDA is the *substrate* (durable graph + IDs + traces + lifecycle). Everything below is a *workflow* that runs on top of it and leaves its result in the graph.

Recipe 1 — Spec Kit scaffolds the feature, AIDA holds the graph

GitHub Spec Kit is excellent at going from a paragraph to a structured `spec.md` → `plan.md` → `tasks.md` for *one* feature. AIDA is built for what happens *after*: keeping those features as a related, queryable, traced graph for the life of the project.

The recipe:

1. Use Spec Kit's `/speckit.specify` to scaffold a feature's `spec.md` under `specs/001-feature/`. Let it do the first-feature heavy lifting.
2. File the feature as an AIDA spec and point it at the artifact: `aida add --title "..." --type story --feature <name>` — then keep the path to `specs/001-feature/spec.md` in the description or an `external_refs` entry.
3. Add the cross-feature edges AIDA exists for — `aida rel add <this> <that> --type references` — the relationships Spec Kit's per-feature directories can't express.

4. From here, code↔spec traces (`// trace:STORY-1`) and the lifecycle (queue → implement → PR → merge → auto-bump) run in AIDA.

The seam, stated honestly: today this bridge is *manual* — you file the AIDA spec and reference the Spec Kit artifact by hand. An opt-in importer that ingests Spec Kit / OpenSpec / Kiro artifacts into AIDA records (preserving paths, assigning stable IDs, inferring relationships conservatively) is filed as **TASK-0416** and not yet shipped. Until it lands, the composition is real but hand-wired; the vs-spec-kit page covers the division of labor in full.

Who this is for: teams that adopted Spec Kit, shipped a handful of features, and have started to feel the per-feature-directory ceiling — cross-feature relationships, rename stability, "what implements this?" across the whole repo.

Recipe 2 — Agent Teams coordinates the burst, AIDA outlives the session

Claude Code's Agent Teams gives you native within-session parallelism: a mailbox, a shared self-claiming task list, dependency auto-unblocking. That is genuinely good *coordination* — for the duration of one session, in one vendor.

AIDA's contribution is **persistence and provenance across sessions and vendors.** The team's task list evaporates when the session ends; AIDA's graph does not.

The recipe:

1. Curate the blessed work in AIDA's queue — `aida queue add <spec>` is the advisor sign-off (only queued + pickable specs are drainable).
2. Drain it: either hand the set to an Agent Team, or run `aida burndown run`, which fans out worktree-isolated implementers over exactly that gated set.
3. Each shipped spec leaves a durable trail AIDA records automatically: `// trace:` comments, the `(SPEC-ID)` commit, and the `aida show` git-linkage block — all of which survive the session the team ran in.

The seam: Agent Teams owns the *live* coordination; AIDA owns the *durable* record. The composition is conceptual today (AIDA doesn't drive an Agent Team through a connector) — but the two are complementary by design, and `aida burndown run` already gives you the same fan-out shape natively when you want it.

Who this is for: anyone who's watched a great parallel agent session produce real work and then wondered, a week later, *what did all that actually change, and why?* AIDA is the answer to the second question.

Recipe 3 — Any MCP editor reads the same graph

AIDA's requirement graph is exposed over MCP, so **any MCP-speaking agent queries the same store** — Claude Code, Codex, Cursor, Continue, Cline, Copilot. The graph is git-canonical and vendor-neutral; the editor is just one more reader.

The recipe:

1. `aida init` writes a `.mcp.json` that registers `aida mcp-serve`.
2. Point your editor's MCP config at it (per-client steps: `docs/agents/aida-mcp-install-matrix.md`).
3. The agent now calls `list_requirements`, `show_requirement`, `search_requirements`, `query_graph`, etc. — reading and writing the same specs your CLI sees.

The seam: there is none worth calling out — this is AIDA working as intended. The point of a vendor-neutral substrate is exactly that the editor doesn't matter. Switch from Cursor to Continue tomorrow and the graph is unchanged.

Who this is for: multi-vendor shops, or anyone who refuses to let their project's intent get locked inside one agent's proprietary memory.

Recipe 4 — `/workflow` orchestrates the task, AIDA tracks the spec

Claude Code Workflows deterministically orchestrate the fan-out *within* a single task — phases, parallel agents, verification gates. AIDA tracks the *spec* that task belongs to, across however many tasks and sessions it takes.

The recipe: run a `/workflow` to execute a complex spec's implementation (decompose → parallel implement → adversarial review → synthesize); the spec it implements still lives in AIDA, picks up its `// trace:` linkage, and moves through the lifecycle on merge. Workflow = how one hard task gets done; AIDA = the spec's life before and after that task.

Recipe 5 — GitHub Issues for the human board, AIDA for the durable graph

If your humans want a live board (Issues, Projects) and your agents want a typed graph, run both and mirror between them.

The recipe: `aida github push <spec>` publishes a spec as a GitHub Issue; `aida github pull` imports Issues back as requirements; `aida github labels --create-missing` sets up the AIDA label taxonomy. Humans get the board they like; the durable, code-linked graph stays in AIDA. The full "when a SaaS PM tool is the right call" split is vs-saas-pm.md.

Recipe 6 — Karpathy-style markdown is the floor; AIDA is the graph on top

If you already keep a structured `REQUIREMENTS.md` an agent can read (the Karpathy approach), you've done the hard part — maintaining the discipline. AIDA doesn't ask you

to throw it away; it adds stable IDs, typed edges, code traces, and an MCP server *on top of* that same habit. The markdown is the floor; the graph is what you get for keeping it.

Recipe 7 — Open protocols carry the handoff; AIDA holds the record

MCP (agent↔tool) and A2A (agent↔agent) are stateless *transports* — A2A by its own spec coordinates agents "without sharing internal memory, state, or tools." AIDA's posture is to **speak** these standards, not replace them: the git-canonical record stays the source of truth, and each protocol is one more way work feeds it.

The recipe: let your agents use A2A (or MCP, or plain git) to hand each other live tasks; land the durable result — what's being worked, by whom, in what state, which code traces back — in AIDA. The transport carries the in-flight handoff; the record survives it. Full division of labor: vs-a2a.md.

The seam, stated honestly: there is no A2A surface in AIDA today — this is a stated posture, not a shipped feature. An A2A message landing as a mailbox item or brief pickup is where a bridge *would* fit, gated on real demand.

The through-line

Pick the layer by its job:

Layer	Job	Lifespan
Spec Kit / OpenSpec / Kiro	Scaffold one feature's spec → plan → tasks	The feature
Agent Teams / `/workflow` / `burndown run`	Coordinate a parallel agent burst	The session / the task
Cursor / Continue / Codex (via MCP)	Read & write specs from your editor	The edit
GitHub Issues / Projects	A live board for humans	The sprint
AIDA	**The durable, vendor-neutral graph of intent + code linkage**	**The project**

Almost every "AIDA vs X" is really "AIDA *with* X." X does a job inside a window of time; AIDA is the substrate that remembers across all of them.

See also

- How AIDA compares — one neighbor at a time.
- When *not* to use AIDA — the honest cases where a neighbor alone is enough.
- aida-mcp-install-matrix — per-client MCP setup for Recipe 3.

- vs-a2a.md — A2A/MCP are transports; AIDA is the record they leave open (Recipe 7).